//HPC-05

```
from mpi4py import MPI
import tensorflow as tf
import numpy as np

# Initialize MPI
comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

# Load dataset
mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize dataset
x_train = x_train / 255.0
x_test = x_test / 255.0

# Reshape for CNN input
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

# Define CNN Model
model = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), activation='relu',
                           input_shape=(28, 28, 1)),
    tf.keras.layers.MaxPooling2D((2,2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(10, activation='softmax')
])

# Training function
def train(model, x_train, y_train, rank, size):

    # Split dataset across MPI nodes
    n = len(x_train)
    chunk_size = n // size

    start = rank * chunk_size
    end = (rank + 1) * chunk_size if rank != size - 1 else n

    x_chunk = x_train[start:end]
    y_chunk = y_train[start:end]

    # Compile model
    model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
```

```
              metrics=['accuracy'])

    # Train model
    model.fit(x_chunk, y_chunk,
            epochs=1,
            batch_size=32,
            verbose=0)

    # Evaluate local training accuracy
    train_loss, train_acc = model.evaluate(x_chunk,
                                   y_chunk,
                                   verbose=0)

    # Aggregate accuracy from all nodes
    train_acc = comm.allreduce(train_acc, op=MPI.SUM)

    return train_acc / size


# Training loop
epochs = 5

for epoch in range(epochs):

    train_acc = train(model,
                 x_train,
                 y_train,
                 rank,
                 size)

    # Evaluate test accuracy
    test_loss, test_acc = model.evaluate(x_test,
                                 y_test,
                                 verbose=0)

    test_acc = comm.allreduce(test_acc, op=MPI.SUM)
    if rank == 0:
        print(f"Epoch {epoch+1}: "
            f"Train accuracy = {train_acc:.4f}, "
            f"Test accuracy = {test_acc/size:.4f}")

//OUTPUT
Epoch 1: Train accuracy = 0.9152, Test accuracy = 0.9278
Epoch 2: Train accuracy = 0.9487, Test accuracy = 0.9561
Epoch 3: Train accuracy = 0.9623, Test accuracy = 0.9664
Epoch 4: Train accuracy = 0.9715, Test accuracy = 0.9728
Epoch 5: Train accuracy = 0.9782, Test accuracy = 0.9789
```